CampusCloud Frontend — API Integration Reference

Team A06 · BDS-8A

This document describes every API endpoint that the frontend dashboard calls (or will call). It is written from the **frontend's perspective** — what we send, what we expect back, and where in the codebase each call lives.

Backend implementation details (routes, middleware, database schema) are intentionally out of scope here.

Base URL

```
http://localhost:5000          ← development default
```

Configured via `VITE_API_URL` in `frontend/.env`. All API modules import it as `import.meta.env.VITE_API_URL`.

Authentication Header

Every protected endpoint requires:

```
Authorization: Bearer <accessToken>
```

The `accessToken` is stored in `AuthContext` (and `localStorage`) after login. It expires after **15 minutes**. On `401`, redirect the user to `/login`.

Endpoints

POST /register

File: `src/api/auth.js → register(email, password)`

Create a new account.

Request body

Request body

```
{
  "email": "student@university.edu",
  "password": "SecurePass1!"
}
```

Validation enforced on the frontend before sending:

- Email must end in `@gmail.com` or `.edu`
- Password must be ≥ 8 characters with uppercase, lowercase, digit, and special character

Success — 201

```
{
  "message": "User registered successfully",
  "accessToken": "<jwt>",
  "user": {
    "id": "uuid",
    "email": "student@university.edu"
  }
}
```

Error responses

Status	Meaning
400	Validation failed or email already in use
500	Server error

POST /login

File: `src/api/auth.js → login(email, password)`

Authenticate an existing user.

Request body

```
{
  "email": "student@university.edu",
  "password": "SecurePass1!"
}
```

Success — 200

```
{
  "accessToken": "<jwt>",
  "user": {
    "id": "uuid",
    "email": "student@university.edu"
  }
}
```

The backend also sets a `refreshToken` HttpOnly cookie (handled automatically by the browser — the frontend does not read it directly).

Error responses

Status	Meaning
400	Missing or malformed fields
401	Wrong email or password
500	Server error

POST /logout

File: `src/api/auth.js → logout()`

Revoke the current session. No request body needed — the backend reads the `refreshToken` cookie automatically.

Success — 200

```
{ "message": "Logged out successfully" }
```

After this call, `AuthContext.logoutCtx()` clears the token and user from state and `localStorage`, then redirects to `/login`.

GET /project

File: `src/api/projects.js → getProjects(token)`

List all projects owned by the logged-in user.

Headers: `Authorization: Bearer <token>`

Success — 200

```
[
  {
    "id": "uuid",
    "name": "web-app",
    "owner_id": "uuid",
    "created_at": "2026-03-22T10:00:00.000Z"
  }
]
```

Returns `[]` if the user has no projects yet.

Error responses

Status	Meaning
401	Missing or invalid JWT
500	Server error

POST /project

File: `src/api/projects.js → createProject(name, token)`

Create a new project.

Headers: `Authorization: Bearer <token>`

Request body

```
{ "name": "my-new-project" }
```

Success — 201

```
{
  "id": "uuid",
  "name": "my-new-project",
  "owner_id": "uuid",
  "created_at": "2026-03-22T10:05:00.000Z"
}
```

Error responses

Status	Meaning
400	`name` field missing
401	Missing or invalid JWT

401	Missing or invalid JWT
500	Server error

Container Endpoints (Mock — Planned)

The following endpoints are **not yet live**. `src/api/instances.js` simulates all of them using `localStorage`. When the BDS-8B data plane is ready, replace each function body with the `fetch()` call shown below.

POST /instance *(planned)*

File: `src/api/instances.js → createInstance(data, token)`

Launch a Docker container.

Request body

```
{
  "name": "nginx-web",
  "image": "nginx:latest",
  "project_id": "uuid",
  "cpu_limit": 1.0,
  "memory_limit_mb": 512
}
```

Expected success — 201

```
{
  "id": "uuid",
  "name": "nginx-web",
  "image": "nginx:latest",
  "project_id": "uuid",
  "status": "running",
  "cpu": 0,
  "memory": 0,
  "created_at": "2026-03-22T10:10:00.000Z"
}
```

Real `fetch()` replacement:

```
const res = await fetch(`${import.meta.env.VITE_API_URL}/instance`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
```

```
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify(data),
});
if (!res.ok) throw new Error('Failed to create instance');
return res.json();
```

DELETE /instance/:id *(planned)*

File: `src/api/instances.js → deleteInstance(id, token)`

Stop and remove a container.

Expected success — 200

```
{ "message": "Instance deleted successfully" }
```

Real `fetch()` replacement:

```
const res = await fetch(`${import.meta.env.VITE_API_URL}/instance/${id}`, {
  method: 'DELETE',
  headers: { Authorization: `Bearer ${token}` },
});
if (!res.ok) throw new Error('Failed to delete instance');
return res.json();
```

GET /instances *(planned)*

File: `src/api/instances.js → getInstances(projectId?, token)`

List containers. Pass `projectId` to filter by project.

Query param (optional): `?project_id=<uuid>`

Expected success — 200

```
[
  {
    "id": "uuid",
    "name": "nginx-web",
    "image": "nginx:latest",
    "project_id": "uuid",
    "status": "running",
    "cpu": 12,
    "memory": 128,
```

```
    "created_at": "2026-03-22T10:10:00.000Z"
  }
]
```

Real `fetch()` replacement:

```
const url = new URL(`${import.meta.env.VITE_API_URL}/instances`);
if (projectId) url.searchParams.set('project_id', projectId);
const res = await fetch(url, {
  headers: { Authorization: `Bearer ${token}` },
});
if (!res.ok) throw new Error('Failed to fetch instances');
return res.json();
```

Frontend Integration Map

Frontend file	Exported function	Endpoint	Status
`src/api/auth.js`	`login()`	`POST /login`	Live
`src/api/auth.js`	`register()`	`POST /register`	Live
`src/api/auth.js`	`logout()`	`POST /logout`	Live
`src/api/projects.js`	`getProjects()`	`GET /project`	Live
`src/api/projects.js`	`createProject()`	`POST /project`	Live
`src/api/instances.js`	`getInstances()`	`GET /instances`	🟡 Mock
`src/api/instances.js`	`createInstance()`	`POST /instance`	🟡 Mock
`src/api/instances.js`	`deleteInstance()`	`DELETE /instance/:id`	🟡 Mock
`src/api/instances.js`	`getStats()`	derived from instances	🟡 Mock

Standard Error Shape

All error responses from the backend follow this format:

```
{
  "message": "Human-readable description of the error",
  "errors": [ ]
}
```

`errors` is an optional array present on validation failures (e.g. password rules on `/register`). The frontend reads `message` to display in the UI.